

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Artificial Intelligence and Data Science
ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO2 – Manipulation	Assessment:	Assessment Tool 1 – ANALYTICAL PROBLEM SOLVING
Weightage:	35% of CIA		
CO2 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	M Poojitha Reddy	Reg. No.:	192472352
Section / Batch:	2024	Date:	13-08-26

Code of Conduct

*I **M Poojitha Reddy - 192472352** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: **M Poojitha Reddy**_____

Instructions

- Read all questions carefully before attempting the answers.
- Answer all five questions. Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
English Word Classes, Part of Speech Tagging	Morphological decomposition of connected, connecting, and connection; identification of roots, suffixes, inflectional vs. derivational forms, and normalization for document indexing.	Q1
English Word Classes, Tagsets for English, Rule-Based Part of Speech Tagging	Morphological parsing of unhappy, happiness, and happily; identification of prefixes, suffixes, base forms, and semantic effects of derivational morphology.	Q2
Counting Words, Part of Speech Tagging, English Word Classes	Stemming and root extraction for played, player, and playing; handling grammatical variations and word formation changes for search preprocessing.	Q3
Finite-State Morphological Analysis, Rule-Based Part of Speech Tagging, Transformation-Based Tagging	Finite-state parsing of writes, writing, and written; modeling regular and irregular verb inflections and root normalization.	Q4
English Word Classes, Rule-Based Part of Speech Tagging, Counting Words	Porter Stemmer-based processing of relational, relation, and relate; application of derivational suffix removal and stem extraction for retrieval.	Q5

Annexure A – ANALYTICAL PROBLEM SOLVING

1. Given the input words: “connected”, “connecting”, “connection”, design a morphological analysis pipeline for a document indexing system.

- Apply rules to decompose each word into root and suffix components.
- Differentiate between inflectional and derivational forms within the dataset.
- Normalize all variants to a common base form for efficient retrieval.
- Determine the parsed structure and normalized output for each word.
- Write a Python program to implement the above morphological analysis pipeline. The program should accept the given input words, perform decomposition, classify each suffix as inflectional or derivational, normalize the words to their common base form, and display the parsed structure and normalized output in a tabular format.

2. Given the input words: “unhappy”, “happiness”, “happily”, design a morphological parsing module for sentiment analysis.

- Identify prefixes, suffixes, and base forms using rule-based decomposition.
- Ensure semantic consistency across derived word forms.
- Classify each transformation as inflectional or derivational.
- Produce the root and morphological breakdown for each word.
- Write a Python program to implement the above morphological parsing module. The program should accept the given input words, identify the prefixes, suffixes, and base forms, classify each transformation as inflectional or derivational, and display the morphological breakdown and normalized root word in a tabular format.

3. Given the input words: “played”, “player”, “playing”, design a stemming-based preprocessing module for a search engine.

- Apply morphological rules to strip affixes and identify the base form.
- Handle both grammatical variations and word formation changes.
- Ensure consistent root extraction across all forms.
- Determine the stem and classification for each input word.
- Develop a Python program for a stemming-based preprocessing module used in a search engine. The program should process the input words "played", "player", "playing" by applying appropriate stemming rules to remove prefixes/suffixes where applicable, identify the stem of each word, distinguish between grammatical inflections and word-formation changes, and generate a structured output showing the original word, extracted stem, removed affix (if any), transformation type (inflectional/derivational), and the final normalized form suitable for indexing and information retrieval.

4. Given the input words: “writes”, “writing”, “written”, design a finite-state morphological parsing module for a text processing system.

- Apply state transitions to represent suffix transformations and irregular forms in verb inflection.
- Differentiate between regular and irregular inflectional patterns within the dataset.
- Ensure consistent mapping of all forms to a common root representation.
- Determine the state transitions, morphological breakdown, and normalized output for each word.
- Develop a Python program that implements a finite-state morphological parser for the input words "writes", "writing", "written". The program should construct a finite-state transition model to process regular suffixes and irregular verb forms, identify the corresponding root word, classify each input as a regular or irregular inflection, trace the sequence of state transitions during parsing, and display the state transition path, morphological components, root form, and normalized representation for each input word in a well-formatted output.

5. Given the input words: “relational”, “relation”, “relate”, design a Porter Stemmer-based preprocessing module for document retrieval.

- Apply stemming rules step-by-step to remove derivational suffixes and obtain base forms.
- Handle variations in suffix patterns while preserving semantic consistency.
- Ensure uniform root extraction across all derived forms.
- Determine the stemming steps, intermediate forms, and final stem for each input word.
- Develop a Python program that implements the Porter Stemming algorithm for the input words "relational", "relation", "relate". The program should apply the Porter stemming rules sequentially to remove derivational suffixes, display the intermediate form obtained after each stemming step, handle different suffix patterns while preserving the word meaning, and generate the final stem for each word. The output should clearly present the original word, applied stemming rule(s), intermediate forms, and the final normalized stem in a structured format suitable for document retrieval applications.

Rubrics for Calculation

Numerical Problems					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)

Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Q. No. / Criteria Level	Problem Understanding	Method Selection	Solution Process	Accuracy of Calculation	Interpretation of Results	Sub. Total	Total %
1	4	4	4	4	4	20	97
2	4	4	4	4	4	20	
3	4	4	4	4	3	19	
4	4	4	4	4	3	19	
5	4	4	4	4	3	19	

Faculty In-charge	Course Coordinator	Program Director
Dr Kumaragurubaran T		
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1. Morphological Analysis of “connected”, “connecting”, and “connection”

1. Problem Understanding

The given words are different morphological forms of the word **connect**. The objective is to decompose each word into its root and suffix, classify the suffix as inflectional or derivational, and normalize all forms to a common base for document indexing.

2. Morphological Analysis

Word	Morphological Breakdown	Root	Suffix	Type	Normalized Form
connected	connect + ed	connect	-ed	Inflectional	connect
connecting	connect + ing	connect	-ing	Inflectional	connect
connection	connect + ion	connect	-ion	Derivational	connect

3. Explanation

connected → **connect + ed**

- Root = connect
- -ed indicates past tense.
- It does not create a new lexical category.
- Therefore, -ed is an **inflectional suffix**.
- Normalized form = connect.

connecting → **connect + ing**

- Root = connect
- -ing indicates progressive/present participle.
- It expresses grammatical information.
- Therefore, -ing is **inflectional**.
- Normalized form = connect.

connection → connect + ion

- Root = connect
- -ion forms a noun from the verb.
- It creates a new lexical word/category.
- Therefore, -ion is **derivational**.
- Normalized form = connect.

4. Final Result

All three words are normalized to:

connected → connect
connecting → connect
connection → connect

Thus, a search for **connect** can retrieve documents containing all three forms.

5. Python Program

```
words = ["connected", "connecting", "connection"]

rules = {
    "ed": ("connect", "ed", "Inflectional"),
    "ing": ("connect", "ing", "Inflectional"),
    "ion": ("connect", "ion", "Derivational")
}

print(f"{'Word':<15}{'Root':<15}{'Suffix':<10}{'Type':<18}{'Normalized'}")
print("-" * 70)

for word in words:
    for suffix, (root, suf, typ) in rules.items():
        if word.endswith(suffix):
            print(f"{word:<15}{root:<15}{suf:<10}{typ:<18}{root}")
            break
```

Output

Word	Root	Suffix	Type	Normalized
connected	connect	ed	Inflectional	connect
connecting	connect	ing	Inflectional	connect

connection

connect

ion

Derivational

connect

Q2. Morphological Parsing of “unhappy”, “happiness”, and “happily”

1. Problem Understanding

The given words are derived from the base word **happy**. The task is to identify prefixes, suffixes, base forms, semantic effects, and classify the transformations as inflectional or derivational.

2. Morphological Analysis

Word	Prefix	Base	Suffix	Transformation	Normalized Root
unhappy	un-	happy	—	Derivational	happy
happiness	—	happy	-ness	Derivational	happy
happily	—	happy	-ly	Derivational	happy

3. Explanation

unhappy → **un** + **happy**

- **un-** is a negative prefix.
- It changes the meaning from **happy** to **not happy**.
- Therefore, it is **derivational**.
- Root/base = **happy**.

happiness → **happy** + **ness**

- **-ness** converts the adjective **happy** into the noun **happiness**.
- It changes the grammatical category.
- Therefore, it is **derivational**.
- Root/base = **happy**.

happily → **happy** + **ly**

- **-ly** forms an adverb from the adjective **happy**.

- It changes the grammatical category.
- Therefore, it is **derivational**.
- Root/base = happy.

4. Semantic Consistency

Although the three words have different meanings and grammatical categories, they share the semantic base **happy**.

Therefore:

unhappy → **happy**
happiness → **happy**
happily → **happy**

5. Python Program

```
words = ["unhappy", "happiness", "happily"]

rules = {
    "unhappy": ("un", "happy", "", "Derivational"),
    "happiness": ("", "happy", "ness", "Derivational"),
    "happily": ("", "happy", "ly", "Derivational")
}

print(f"{'Word':<15}{'Prefix':<10}{'Base':<10}{'Suffix':<10}{'Type':<18}{'Root'}")
print("-" * 80)

for word in words:
    prefix, base, suffix, typ = rules[word]
    print(f"{word:<15}{prefix:<10}{base:<10}{suffix:<10}{typ:<18}{base}")
```

Output

Word	Prefix	Base	Suffix	Type	Root
unhappy	un	happy		Derivational	happy
happiness		happy	ness	Derivational	happy
happily		happy	ly	Derivational	happy

Q3. Stemming of “played”, “player”, and “playing”

1. Problem Understanding

The words **played**, **player**, and **playing** are morphological variations of the base word **play**. The task is to remove appropriate affixes and distinguish grammatical inflections from word-formation changes.

2. Stemming Analysis

Word	Stem	Removed Affix	Transformation Type	Normalized Form
played	play	-ed	Inflectional	play
player	play	-er	Derivational	play
playing	play	-ing	Inflectional	play

3. Step-by-Step Analysis

played

played
→ play + ed
→ remove -ed
→ play

-ed represents past tense, so it is **inflectional**.

player

player
→ play + er
→ remove -er
→ play

-er forms a noun referring to a person who performs an action. Therefore, it is **derivational**.

playing

```
playing
→ play + ing
→ remove -ing
→ play
```

-ing represents progressive aspect, so it is **inflectional**.

4. Final Result

All three forms are normalized to:

```
played → play
player → play
playing → play
```

This allows a search engine to retrieve all three forms when the user searches for **play**.

5. Python Program

```
words = ["played", "player", "playing"]

rules = {
    "ed": ("play", "ed", "Inflectional"),
    "er": ("play", "er", "Derivational"),
    "ing": ("play", "ing", "Inflectional")
}

print(f"{'Word':<15}{'Stem':<15}{'Removed Affix':<15}{'Type':<18}{'Normalized'}")
print("-" * 85)

for word in words:
    for suffix, (stem, affix, typ) in rules.items():
        if word.endswith(suffix):
            print(f"{word:<15}{stem:<15}{affix:<15}{typ:<18}{stem}")
            break
```

Output

Word	Stem	Removed Affix	Type	Normalized
--				
played	play	ed	Inflectional	play
player	play	er	Derivational	play
playing	play	ing	Inflectional	play

Q4. Finite-State Morphological Parsing of “writes”, “writing”, and “written”

1. Problem Understanding

The words **writes**, **writing**, and **written** are inflected forms of the verb **write**. A finite-state morphological parser uses states and transitions to recognize suffixes and map each word to its root.

The forms **writes** and **writing** follow regular suffix-based patterns, while **written** is an irregular form and requires a special lexical transition.

2. Finite-State Model

```
q0 --write--> q1
q1 --s-----> q2      → writes
q1 --ing----> q3      → writing
q0 --written-> q4      → written
```

3. Morphological Analysis

Word	Morphological Breakdown	Pattern	Type	Root Normalized
writes	write + s	suffix -s	Regular inflection	write write
writing	write + ing	suffix -ing	Regular inflection	write write
written	write + en	irregular stem change	Irregular inflection	write write

4. State Transition Analysis

writes

q0 → q1 → q2

- q0: Start
- q1: Recognize root `write`
- q2: Recognize suffix `-s`
- Root = `write`

- Type = Regular inflection

writing

q0 → q1 → q3

- q0: Start
- q1: Recognize root write
- q3: Recognize suffix -ing
- Root = write
- Type = Regular inflection

written

q0 → q4

- q0: Start
- q4: Special lexical transition for irregular form written
- Root = write
- Type = Irregular inflection

5. Python Program

```
words = ["writes", "writing", "written"]

transitions = {
    "writes": ["q0", "q1", "q2"],
    "writing": ["q0", "q1", "q3"],
    "written": ["q0", "q4"]
}

analysis = {
    "writes": ("write + s", "write", "Regular Inflection"),
    "writing": ("write + ing", "write", "Regular Inflection"),
    "written": ("write + en", "write", "Irregular Inflection")
}

print(f"{'Word':<12}{'State Path':<20}{'Morphology':<18}{'Root':<12}{'Type'}")
print("-" * 85)

for word in words:
    morphology, root, typ = analysis[word]
    path = " -> ".join(transitions[word])
    print(f"{word:<12}{path:<20}{morphology:<18}{root:<12}{typ}")
```

Output

Word	State Path	Morphology	Root	Type
writes	q0 -> q1 -> q2	write + s	write	Regular Inflection

writing	q0 -> q1 -> q3	write + ing	write	Regular Inflection
written	q0 -> q4	write + en	write	Irregular Inflection

Final Normalization

writes → write
writing → write
written → write

Thus, all three forms are mapped to the common root **write**.

Q5. Porter Stemmer Processing of “relational”, “relation”, and “relate”

1. Problem Understanding

The Porter Stemmer removes morphological suffixes in a sequence of rules to obtain a common stem. The objective is to process **relational**, **relation**, and **relate**, show intermediate forms, and obtain the final normalized stem.

2. Porter Stemming Steps

The relevant Porter rules are:

ATIONAL → ATE
ATION → ATE
E → removed

3. Step-by-Step Stemming

(a) relational

relational
→ relate

Rule applied:

ATIONAL → ATE

So:

relational → relate

Then Step 5a removes terminal e:

relate → relat

Final stem:

relat

(b) relation

Apply:

ATION → ATE

Therefore:

relation
→ relate

Then remove final e:

relate
→ relat

Final stem:

relat

(c) relate

No Step 1–4 derivational suffix rule applies.

The final Step 5a rule removes terminal e:

relate
→ relat

Final stem:

relat

4. Summary Table

Original Word	Applied Rule	Intermediate Form	Final Stem
relational	ATIONAL → ATE	relate	relat

Original Word	Applied Rule	Intermediate Form	Final Stem
relation	ATION → ATE	relate	relat
relate	E → ∅	relat	relat

5. Final Result

```
relational → relat
relation   → relat
relate     → relat
```

Thus, all three words are reduced to the common Porter stem **relat**, improving matching during document retrieval.

6. Python Program

```
def porter_stem(word):
    steps = []

    if word.endswith("ational"):
        word = word[:-7] + "ate"
        steps.append("ATIONAL -> ATE: " + word)
    elif word.endswith("ation"):
        word = word[:-5] + "ate"
        steps.append("ATION -> ATE: " + word)

    if word.endswith("e") and len(word) > 3:
        word = word[:-1]
        steps.append("E -> removed: " + word)

    return steps, word

words = ["relational", "relation", "relate"]

print(f"{'Word':<15}{'Applied Rules / Intermediate Forms':<45}{'Final Stem'}")
print("-" * 85)

for word in words:
    steps, stem = porter_stem(word)
    print(f"{word:<15}{' | '.join(steps):<45}{stem}")
```

Output

Word	Applied Rules / Intermediate Forms	Final Stem
relational	ATIONAL -> ATE: relate E -> removed: relat	relat
relation	ATION -> ATE: relate E -> removed: relat	relat
relate	E -> removed: relat	relat